

JavaScript for Testers

A practical guide for QA, Automation Engineers and SDETs

JavaScript fundamentals → Browser DevTools → DOM → Test automation →
Playwright foundations

Recommended tools: Chrome DevTools, VS Code, Node.js and Playwright

Learning Roadmap

This guide focuses on the JavaScript that testers actually use when inspecting web applications and building automation.

1	JavaScript essentials	Variables, types, operators, conditions
2	Arrays & objects	Test data and application data
3	Functions	Reusable validation and utility logic
4	Modern JavaScript	let/const, arrow functions, template literals
5	DOM & browser	Elements, attributes, text and values
6	Events & forms	Clicks, input, submit and validation
7	DevTools	Console, selectors and XPath
8	Async JavaScript	Promises, async/await and waits
9	Test automation	Assertions, reusable helpers and data
10	Playwright foundation	Locators, pages, API and fixtures

1. JavaScript Essentials for Testers

1. Variables

Use `let` for values that change and `const` for values that should not be reassigned.

```
const baseUrl = "https://example.com";  
let retryCount = 0;
```

2. Data Types

Understand the common values you will see in test data and APIs.

```
let username = "tester";  
let age = 30;  
let active = true;  
let roles = ["admin", "tester"];  
let user = { name: "Bala", active: true };
```

3. Comparison

Use strict comparison in assertions and validation.

```
console.log(10 === 10);  
console.log("10" === 10);  
console.log(10 !== 20);
```

Tester note: Prefer `===` and `!==` in test automation to avoid unexpected type conversion.

4. Conditions

Write validation logic based on expected application behavior.

```
if (status === "PASS") {  
  console.log("Test passed");  
} else {  
  console.log("Test failed");  
}
```

2. Arrays, Objects & Test Data

5. Array Operations

Use arrays to store multiple test values.

```
const users = ["admin", "manager", "customer"];
console.log(users[0]);
console.log(users.length);
```

6. Array Search

Find whether a required test value exists.

```
console.log(users.includes("admin"));
```

7. map()

Transform test data into a format needed by automation.

```
const names = ["John", "Mary", "Sam"];
const upper = names.map(name => name.toUpperCase());
console.log(upper);
```

8. filter()

Select only test data that meets a condition.

```
const users = [
  {name:"A", active:true},
  {name:"B", active:false}
];
const activeUsers = users.filter(u => u.active);
```

9. Object Properties

Read and update application/test data.

```
const customer = {name:"Bala", age:30};
console.log(customer.name);
customer.age = 31;
```

3. Functions & Reusable Test Utilities

10. Validation Function

Create reusable functions instead of repeating test logic.

```
function validateStatus(actual, expected) {  
  return actual === expected;  
}  
console.log(validateStatus("PASS", "PASS"));
```

11. Arrow Function

Use concise functions commonly seen in modern automation code.

```
const add = (a, b) => a + b;  
console.log(add(10, 20));
```

12. Reusable Test Data Function

Return a new user object whenever a test needs test data.

```
function createUser(name, role) {  
  return { name, role, active: true };  
}
```

13. Destructuring

Extract values from test data objects cleanly.

```
const user = {name:"Bala", role:"tester"};  
const {name, role} = user;  
console.log(name, role);
```

4. DOM & Browser Testing

14. Find by ID

Use JavaScript in Chrome DevTools to inspect a web element.

```
const button = document.getElementById("login");
console.log(button);
```

15. CSS Selector

Find elements using CSS selectors.

```
document.querySelector("button.login");
document.querySelectorAll("a");
```

16. Read Text

Print the visible text of an element.

```
const heading = document.querySelector("h1");
console.log(heading.innerText);
```

17. Read Input Value

Read a textbox value.

```
const email = document.querySelector("#email");
console.log(email.value);
```

18. Attribute

Read an element attribute such as href or aria-label.

```
const link = document.querySelector("a");
console.log(link.getAttribute("href"));
```

19. Find All Links

Print every link's text and URL.

```
document.querySelectorAll("a").forEach((link, i) => {
  console.log(`${i + 1}. ${link.innerText} → ${link.href}`);
})
```

5. XPath & Chrome DevTools

20. XPath Lookup

Use Chrome DevTools `$x()` to test XPath expressions.

```
$x('//button')
```

21. XPath by Text

Find a link whose text contains Login.

```
$x("//a[contains(normalize-space(.), 'Login')]")
```

22. XPath Attribute

Find an element using an attribute.

```
$x("//input[@name='username']")
```

23. Count Matching Elements

Check how many elements match a selector.

```
console.log(document.querySelectorAll("button").length);
```

24. Inspect a Button

Print the text and disabled state of the first button.

```
const b = document.querySelector("button");  
console.log(b.innerText);  
console.log(b.disabled);
```

6. Events, Forms & Validation

25. Click Event

Attach a JavaScript click handler.

```
document.querySelector("#login").addEventListener("click", () => {  
  console.log("Login clicked");  
});
```

26. Input Event

Monitor text entered into a textbox.

```
document.querySelector("#username").addEventListener("input", e => {  
  console.log(e.target.value);  
});
```

27. Mandatory Validation

Display a validation message when a required field is empty.

```
if (!username.value.trim()) {  
  console.log("All fields are mandatory");  
}
```

28. Form Submit

Prevent the browser from submitting and inspect the data.

```
form.addEventListener("submit", e => {  
  e.preventDefault();  
  console.log("Form submitted");  
});
```

7. Async JavaScript for Testers

29. Promise

Understand the basic object used to represent future completion.

```
const result = new Promise(resolve => {
  setTimeout(() => resolve("Done"), 1000);
});
```

30. async / await

Write asynchronous code in a readable style.

```
async function getData() {
  const response = await fetch("/api/users");
  const data = await response.json();
  console.log(data);
}
```

31. Fetch API

Call an API from the browser console.

```
fetch("https://example.com/api/users")
  .then(response => response.json())
  .then(data => console.log(data));
```

Tester note: In real projects, authentication, CORS and environment URLs must be handled correctly.

8. JavaScript Concepts Used in Playwright

32. Locator Thinking

Understand that automation code stores references to elements and performs actions.

```
const button = page.getByRole("button", { name: "Login" });  
await button.click();
```

33. Assertions

Compare actual application state with the expected result.

```
await expect(page.getByText("Welcome")).toBeVisible();
```

34. Loops for Test Data

Run the same validation for multiple values.

```
for (const value of ["admin", "manager", "customer"]) {  
  console.log(value);  
}
```

35. Array of Test Cases

Represent multiple scenarios as structured JavaScript data.

```
const cases = [  
  {input:"admin", expected:"Admin"},  
  {input:"user", expected:"User"}  
];
```

9. Hands-On Tester Exercises

36. Login Form Inspector

Find username, password and Login button. Print their tag names, text/value and relevant attributes.

Your task: write the code, test it in DevTools/Node.js, and record the expected and actual result.

37. Link Audit

Print every link on a page with index, visible text and href. Identify empty-text links.

Your task: write the code, test it in DevTools/Node.js, and record the expected and actual result.

38. Button Audit

Find all buttons and report text, disabled state and type.

Your task: write the code, test it in DevTools/Node.js, and record the expected and actual result.

39. Form Validator

Build a Name + Age form. Empty fields must display 'All fields are mandatory'.

Your task: write the code, test it in DevTools/Node.js, and record the expected and actual result.

40. Dynamic Table

Find all table rows, print cell values and identify rows matching a condition.

Your task: write the code, test it in DevTools/Node.js, and record the expected and actual result.

41. Test Data Filter

Create 10 user objects and filter active users, then map their names.

Your task: write the code, test it in DevTools/Node.js, and record the expected and actual result.

42. API Data Check

Use fetch() to retrieve JSON and validate that required properties exist.

Your task: write the code, test it in DevTools/Node.js, and record the expected and actual result.

43. Mini Automation Utility

Create functions for login data, status validation and test-result formatting.

Your task: write the code, test it in DevTools/Node.js, and record the expected and actual result.

10. Quick Reference for Testers

Need to...	JavaScript
Print value	<code>console.log(value)</code>
Find by ID	<code>document.getElementById('id')</code>
Find one CSS element	<code>document.querySelector('selector')</code>
Find many CSS elements	<code>document.querySelectorAll('selector')</code>
Find XPath in Chrome	<code>\$x('//selector')</code>
Get visible text	<code>element.innerText</code>
Get input value	<code>element.value</code>
Get attribute	<code>element.getAttribute('name')</code>
Count elements	<code>document.querySelectorAll('a').length</code>
Loop array	<code>items.forEach(item => console.log(item))</code>
Filter array	<code>items.filter(item => condition)</code>
Transform array	<code>items.map(item => transform(item))</code>
Wait for async result	<code>await promise</code>

Final Goal

A tester who understands JavaScript can inspect web pages, debug automation, manipulate test data, validate application behavior, work with asynchronous APIs, and read/write Playwright or Selenium automation more confidently.